

Análise comparativa de algoritmos com uso de paralelismo

Link do GitHub: substituir este campo pelo link final do repositório.

Resumo

Este projeto compara três abordagens para contagem de uma palavra em arquivos de texto usando Java:

- `SerialCPU`: percorre o texto em uma única thread.
- `ParallelCPU`: divide o texto em blocos e usa um pool de threads.
- `ParallelGPU`: usa OpenCL via JOCL para avaliar as posições do texto em paralelo.

O programa executa cada método pelo menos três vezes em cada amostra, registra os tempos em CSV e gera gráficos SVG para apoiar a análise.

Estrutura do Projeto

```
.
├── lib/
│   └── jocl-2.0.4.jar
├── results/
│   ├── resultados.csv
│   └── graficos/
├── scripts/
│   └── run.sh
├── src/
│   └── Main.java
├── README.md
└── README.pdf
```

Introdução

A contagem de palavras foi escolhida por ser uma operação simples, repetitiva e fácil de validar entre implementações seriais e paralelas. O objetivo é observar quando o paralelismo compensa o custo de gerenciamento de threads ou de transferência para OpenCL.

A implementação considera uma ocorrência quando a palavra aparece como palavra inteira, ignorando diferença entre letras maiúsculas e minúsculas. Caracteres alfabéticos `a-z` e dígitos `0-9` são tratados como parte da palavra; pontuação, espaços e quebras de linha são tratados como separadores.

Metodologia

Foram usadas três amostras de texto do diretório `~/Downloads/Amostras`:

- `DonQuixote-388208.txt`
- `Dracula-165307.txt`

- `MobyDick-217452.txt`

Cada arquivo é carregado uma vez em memória e normalizado para letras minúsculas. O tempo medido corresponde ao processamento da contagem, sem incluir leitura do arquivo.

Antes das repetições registradas, o programa executa um aquecimento não gravado no CSV para reduzir distorções da primeira chamada do JIT da JVM.

A execução padrão realiza:

- 3 repetições por arquivo.
- 1 execução serial por repetição.
- Execuções paralelas em CPU variando o número de threads.
- Execução OpenCL via JOCL quando houver plataforma OpenCL disponível.
- Registro dos resultados em `results/resultados.csv`.
- Geração dos gráficos em `results/graficos/`.

A análise estatística principal usa a média dos tempos por método, por arquivo e por quantidade de threads no caso do `ParallelCPU`.

Como Executar

Requisito principal:

- Java 21 ou superior.
- Biblioteca `jocl-2.0.4.jar`.
- Driver/runtime OpenCL instalado para executar de fato em GPU.

O script procura a biblioteca nesta ordem:

- Variável de ambiente `JOCL_JAR`.
- `lib/jocl-2.0.4.jar`.
- `~/Downloads/jocl-2.0.4.jar`.

Execução padrão com as amostras:

```
./scripts/run.sh
```

Por padrão, o programa procura os textos em `~/Downloads/Amostras`. Se as amostras estiverem em outro diretório, informe o caminho:

```
./scripts/run.sh --samples /caminho/para/Amostras
```

Execução escolhendo palavra, repetições e threads:

```
./scripts/run.sh --word whale --runs 3 --threads 1,2,4,8
```

Execução simples em um único arquivo:

```
./scripts/run.sh --input ~/Downloads/Amostras/Dracula-165307.txt --word blood --
```

```
threads 4
```

Execucao sem OpenCL/GPU:

```
./scripts/run.sh --no-gpu
```

Compilacao manual, se nao quiser usar o script:

```
mkdir -p out
javac -encoding UTF-8 -cp lib/jocl-2.0.4.jar -d out src/Main.java
java -cp out:lib/jocl-2.0.4.jar Main
```

No Linux, prefira `./scripts/run.sh`, pois ele tambem cria automaticamente um symlink local `native/libOpenCL.so` quando o sistema possui apenas `libOpenCL.so.1`.

Execucao pelo IntelliJ IDEA

Se executar pelo IntelliJ IDEA e aparecer erro relacionado a `libOpenCL.so`, a forma mais simples e rodar pelo terminal com:

```
./scripts/run.sh
```

Se quiser rodar pelo botao **Run** do IntelliJ, configure a execucao em `Run > Edit Configurations...`:

- `Main class`: `Main`
- `Working directory`: diretorio raiz do projeto
- `Environment variables`:
`LD_LIBRARY_PATH=/caminho/sem/espacos/para/native`
- `VM options`:
`-Djava.library.path=/caminho/sem/espacos/para/native`

Observacao: se o caminho do projeto tiver espacos, como `Area de Trabalho`, prefira criar ou usar um diretorio auxiliar sem espacos para a pasta `native`. Caminhos com espacos podem fazer o Java interpretar parte do caminho como se fosse o nome da classe principal.

Resultados e Discussao

Os resultados ficam em `results/resultados.csv`, com as colunas:

- `sample`: arquivo de entrada.
- `file_size_bytes`: tamanho do arquivo.
- `word`: palavra pesquisada.
- `method`: metodo executado.
- `workers`: quantidade de threads ou identificador OpenCL.
- `run`: numero da repeticao.
- `occurrences`: quantidade encontrada.

- **time_ms**: tempo em milissegundos.
- **device**: dispositivo usado.
- **status**: OK, SKIPPED ou ERROR.

Graficos gerados:



Medias obtidas na execucao local com a palavra **the**:

Arquivo	Metodo	Threads	Ocorrencias	Tempo medio (ms)
DonQuixote-388208.txt	SerialCPU	1	188	16.751
DonQuixote-388208.txt	ParallelCPU	1	188	21.335
DonQuixote-388208.txt	ParallelCPU	2	188	13.480
DonQuixote-388208.txt	ParallelCPU	4	188	11.994
Dracula-165307.txt	SerialCPU	1	8104	8.306
Dracula-165307.txt	ParallelCPU	1	8104	13.634
Dracula-165307.txt	ParallelCPU	2	8104	9.939
Dracula-165307.txt	ParallelCPU	4	8104	6.483
MobyDick-217452.txt	SerialCPU	1	14727	11.146
MobyDick-217452.txt	ParallelCPU	1	14727	10.734
MobyDick-217452.txt	ParallelCPU	2	14727	8.214
MobyDick-217452.txt	ParallelCPU	4	14727	8.048

Na maquina usada para esta execucao local nao havia plataforma OpenCL registrada em `/etc/OpenCL/vendors`, entao as 9 execucoes **ParallelGPU** foram registradas como **SKIPPED**. O codigo OpenCL esta implementado em `src/Main.java`; para obter tempos reais de GPU, execute o mesmo comando em uma maquina com driver OpenCL instalado.

Interpretacao esperada:

- Em arquivos pequenos, o `SerialCPU` pode vencer porque nao paga custo de criacao e sincronizacao de threads.
- Em arquivos maiores, o `ParallelCPU` tende a reduzir o tempo quando ha nucleos disponiveis e a divisao de trabalho compensa.
- O `ParallelGPU` pode ser mais lento em textos pequenos ou medios porque ha custo de criacao de buffers, transferencia de memoria e chamada do kernel OpenCL.
- Se a maquina nao tiver GPU ou runtime OpenCL configurado, o programa marca a execucao OpenCL como `SKIPPED` para manter o CSV valido.

Conclusao

O trabalho mostra que paralelismo nao garante ganho automatico. A versao serial e simples e pode ser eficiente para entradas menores. A versao paralela em CPU e a alternativa mais equilibrada para textos maiores quando o numero de threads e ajustado ao processador. A versao OpenCL/GPU depende fortemente do ambiente de execucao e tende a valer mais quando o volume de dados e grande o suficiente para compensar os custos de transferencia e inicializacao.

Referencias

- Documentacao Java: `ExecutorService`, `Callable`, `Future`, `Files`.
- JOCL 2.0.4: bindings Java para OpenCL.
- OpenCL: modelo de execucao com kernels e work-items.
- Project Gutenberg: origem dos textos usados como amostras.

Anexos

Codigo principal

Arquivo: `src/Main.java`

O codigo contem:

- Metodo `serialCPU(byte[] text, byte[] word)`.
- Metodo `parallelCPU(byte[] text, byte[] word, int threads)`.
- Metodo `parallelGPU(OpenCLCounter counter, byte[] text, byte[] word)`.
- Classe `OpenCLCounter`, responsavel pela integracao JOCL/OpenCL.
- Geracao de CSV e graficos SVG.

Biblioteca externa

A biblioteca `jocl-2.0.4.jar` fica no diretorio `lib/`, que e o local correto para dependencias

externas deste projeto. Para executar, deixe o arquivo em um destes locais:

- `lib/jocl-2.0.4.jar`
- `~/Downloads/jocl-2.0.4.jar`

Ou execute informando o caminho:

```
JOCL_JAR=/caminho/para/jocl-2.0.4.jar ./scripts/run.sh
```